

2.1 Pipes

Read the pipe as then. Chain steps instead of nesting calls or leaving extra objects in the environment.

1. Read the pipe as "then"

Note 1.2 introduced the pipe. This note is about using it for a sequence of steps.

You can do several things to the same data by:

- creating intermediate objects
- nesting functions
- piping the result of one function into the next

Read either pipe as **then**: take what is on the left, *then* do what follows.

```
x |> mean()
```

is the same idea as `mean(x)`. The gain appears when there are several steps.

In this course you may use either pipe:

```
|>      # Base R (R 4.1 and later). No extra package.
%>%    # magrittr / tidyverse. Comes with library(tidyverse).
```

They are interchangeable for the work we do this week. The Base R pipe `|>` only fills the **first** argument of the next function. The magrittr pipe `%>%` can place the left-hand result elsewhere with `.` if you need a later argument:

```
x %>% mean(na.rm = TRUE)
x %>% mean(., na.rm = TRUE)  # same thing; the dot is the piped value
```

We will not need the dot this week. Put extra arguments in the function that uses them, as in section 3.

In RStudio, **Ctrl+Shift+M** (Windows) or **Cmd+Shift+M** (Mac) inserts a pipe with spaces around it. After a pipe, Enter indents the next line.

You can turn the shortcut toward `|>` or `%>%` under **Tools** → **Global Options...** → **Code**.

2. Three ways to compute a geometric mean

The [geometric mean](#) is useful for positive, skewed data, ratios, and growth rates. A numerically stable recipe is:

1. take the log of each value
2. average those logs
3. exponentiate the average

Additions of logs are usually more stable than a long product. A small illustration:

```
a <- sqrt(2)
a^2 == 2
# [1] FALSE

exp(log(a) + log(a)) == 2
# [1] TRUE
```

Generate some positive numbers so we can compare styles. `set.seed()` makes `rnorm()` repeatable.

```
set.seed(10)
x <- abs(rnorm(100))
head(x)
# [1] 0.01874617 0.18425254 1.37133055 0.59916772 0.29454513 0.38979430
```

`rnorm()` draws from a normal distribution. `abs()` makes the draws non-negative so `log()` is defined.

Intermediate objects

```
log_x <- log(x)
mean_lx <- mean(log_x)
geo_mean <- exp(mean_lx)
geo_mean
# [1] 0.5703996
```

This is easy to debug one line at a time. The cost is extra names in the environment, especially if you repeat the calculation.

Nested calls

```
geo_mean <- exp(mean(log(x)))
geo_mean
# [1] 0.5703996
```

Fine for two or three functions. Past that, the parentheses become hard to read and hard to edit.

A pipe

```
x |>
  log() |>
  mean() |>
  exp()
# [1] 0.5703996
```

One task per line. To keep the result:

```
geo_mean <- x |>
  log() |>
  mean() |>
  exp()
```

Some people assign at the end with `->`. Either is fine. Left assignment at the top is the usual style in this course, because you can run the pipeline line by line without creating `geo_mean` until you want it.

Load tidyverse if you want `%>%` instead of `|>`:

```
library(tidyverse)
```

3. Extra arguments stay in the function that needs them

The piped value usually becomes the **first** argument of the next function. Other arguments stay written in that call:

```
geo_mean <- x |>
  log() |>
  mean(na.rm = TRUE) |>
  exp()
```

That is `mean(log(x), na.rm = TRUE)`, then `exp(...)`.

A pipe helps you:

- read the work top to bottom instead of inside-out
 - avoid a nest of parentheses
 - add, comment, or drop a step without renaming intermediates
-

4. `subset()` keeps the rows you want

`subset()` is Base R. Give it a data frame and a condition written with column names. It returns the rows where the condition is `TRUE`.

```
subset(iris, Sepal.Length > mean(Sepal.Length))
```

That is the same idea as indexing, but you do not have to repeat the data name:

```
iris[iris$Sepal.Length > mean(iris$Sepal.Length), ]
```

The optional `select` argument keeps columns. You can name them, or drop some with `-`:

```
subset(iris, Sepal.Length > 5, select = c(Sepal.Length, Species))
subset(iris, Sepal.Length > 5, select = -c(Sepal.Width, Petal.Width))
```

`subset()` also works on a vector. The condition is written in terms of the vector itself:

```
x <- c(2, 9, 4, 11)
subset(x, x > 5)
# [1] 9 11
```

Rows (or values) where the condition is `NA` are dropped. `?subset` says the function is convenient for interactive work. In a function you will keep, prefer `[]` or the dplyr verbs below. Non-standard evaluation of column names is the reason.

The closest dplyr functions

`dplyr` splits the two jobs `subset()` can do in one call:

- `filter()` is the closest match to the row condition (the second argument of `subset()`)
- `select()` is the closest match to `select = ...`

```
library(dplyr)

iris |>
  filter(Sepal.Length > mean(Sepal.Length))

iris |>
  filter(Sepal.Length > 5) |>
  select(Sepal.Length, Species)
```

`filter()` and `select()` are the tidyverse versions we will use later. You can still read `subset()` when you see it, including in the next example.

5. Other magrittr pipes (aside)

`magrittr` also has `%\%$`, which exposes column names on the right-hand side. You may see it in older code. We will stay with `|>` or `%>%`.

The next pipeline first uses `subset()` to keep the longer sepals, then `%%` so `cor()` can see `Sepal.Length` and `Sepal.Width` as names rather than as `iris$...`. `dplyr::filter()` would do the same row step. `dplyr::select()` plus `cor()` on two columns, or `dplyr::summarize(cor = cor(Sepal.Length, Sepal.Width))` after `filter()`, is the tidyverse version of the whole thing.

```
library(magrittr)
iris |>
  subset(Sepal.Length > mean(Sepal.Length)) %$%
  cor(Sepal.Length, Sepal.Width)
# [1] 0.3361992
```

6. Practice

Using a pipe, and without intermediate objects:

1. draw 100 values from a normal distribution with standard deviation 10
2. sort them
3. take lagged differences
4. average those differences
5. round the average to one decimal place

Useful functions: `rnorm()`, `sort()`, `diff()`, `mean()`, `round()`. Use `set.seed()` if you want a repeatable number.